



# Implementation of SSL/TLS Security with MQTT Protocol in IoT Environment

Iqbal Luqman Bin Mohd Paris<sup>1</sup> · Mohamed Hadi Habaebi<sup>1</sup> ·  
Alhareth Mohammed Zyoud<sup>2</sup>

Accepted: 9 July 2023 / Published online: 25 July 2023

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2023

## Abstract

Internet of Things (IoT) is the interconnection of devices with the internet to deliver its tasks. Nowadays, security is the main concern relating to these devices. Low in power storage, low in processing capabilities and low in data storage make it hard to provide a strong set of security protocols to protect the vulnerable devices “things”. Having internet as its backbone, allows the devices to communicate seamlessly. However, without any form of protection, it would open the door for hackers or middleman to hijack the connection, steal data and sabotage the information. In this paper, Secure Socket Layer and Transport Layer Security (SSL/TLS) protocol is implemented on top of Message Queuing Telemetry Transport (MQTT) IoT application protocol and the performance of the network is evaluated and analyzed in a typical IoT testbed comprising Raspberry Pi4 and ESP32 nodes. This work focuses on energy consumption, generated overhead, system complexity and required data storage resources. Experimental results of stress testing the system indicates that SSL/TLS encryption, operating with MQTT Quality of Service (QoS) level 2, while increasing the traffic rate 3.5 orders of magnitude yields more than two thousand times the amount of overhead generated and results in 73.25 J of consumed energy. Whereas operating without the SSL/TLS encryption under the same stress testing conditions yields only 140 times the amount of overhead generated and results in a mere 18.76 J of consumed energy. This difference of 4 folds on consumed energy indicates that the SSL/TLS -enabled node battery can only last a quarter of the lifespan of the TLS-free node and concluding the SSL/TLS encryption is not a viable solution for battery-operated IoT nodes.

**Keywords** IoT · Protocols · MQTT · TLS/SSL · Security · Performance analysis · Energy harvesting

---

✉ Mohamed Hadi Habaebi  
habaebi@iiu.edu.my

<sup>1</sup> Department of Electrical and Computer Engineering, International Islamic University Malaysia, Kuala Lumpur, Malaysia

<sup>2</sup> Department of Electrical and Computer Engineering, Birzeit University, Birzeit, Ramallah, Palestine

## 1 Introduction

Internet of thing (IoT) comprises a collection of devices that provide services of transferring data from the sensor to the storage or application layer with minimum interaction of human or machine during the process. The task of transferring data is driven by the internet. As the internet is involved during the process, a strong set of security should be implemented in order to avoid any data breach or attack. The lack of storage capacity, low processing capability and low energy storage of internet of things devices have been the major drawback in implementing the best security for the IoT devices. Therefore, to provide the security for the internet of things, there are factors that should be considered which are the security system should be low in power consumption, required low processing capacity and low in overhead. Furthermore, few papers in the open literature, have attempted to quantify the effect of implementing these security measures on IoT device's performance [1–4].

The main contributions of this paper are as follows:

- The implementation of Secure Socket Layer/ Transport Layer Security (SSL/TLS) protocol with Message Queuing Telemetry Transport (MQTT) protocol in a typical IoT testbed comprising Raspberry Pi 4 and ESP32 nodes.
- The quantifying analysis of system overhead, energy consumption, memory storage consumption and execution time due to the implementation of SSL/TLS with MQTT, system complexity and overhead exchanged, and data storage resources required.

The rest of this paper is organized as follows: the theoretical background and the related works are presented in Sect. 2. Section 3 explains the methodology and testbed design. Results are discussed in Sect. 4. In Sect. 5, future works are suggested. A conclusion is drawn in Sect. 6.

## 2 Background Theory and Related Works

In this section, security of IoT is discussed. Then, the architecture of IoT, encryption methods available to protect the IoT environment, and application layer protocols are explained in details.

### 2.1 Security in IoT

As technology is involving across the globe, there is no doubt that IoT devices would become integral part in human life. During the last ten years the number of connected devices to the internet have increased rapidly. From 2 billion connected devices in 2013, the number is increased to 15 billion by 2023, and it is expected to grow to 30 billion devices by 2030 [1, 2]. These statistics are enough to prove the exponential increase in number of IoT devices in the near future. With involvement of internet in the operating system of IoT devices, there is a high probability that it can be the target for series of cyber-attacks such as denial of service, replay attack, man in the middle, cloning of things, eavesdropping and routing attacks [3]. The facts that IoT device is low in power resources, processing capabilities and memory capacity makes it hard to implement a complex and secure security system to protect the IoT as this type of security usually requires a large bandwidth, high overhead and high energy consumption [4]. To secure an IoT device, it is

more suitable to provide a peer security for IoT system layer by layer, and one way to do that is to encrypt network traffic.

## 2.2 IoT Architecture

There are various different architectures models utilized in the previous studies to illustrate the setup of IoT. One of the common is five-layer architecture model which consist of perceptual layer, application layer, transport layer, processing layer, and business layer. The perceptual layer is where the sensor, Radio Frequency Identification (RFID) tags are located. According to [5], all hardware related to IoT devices belong to this layer. Sensors, reader, RFID Tags are all work as a data reader from the surrounding. After that, this data will be transported by transport layer to another process. Next layer is the application layer responsible for delivering specific services to the user. MQTT, Constrained Application Protocol (CoAp) and HyperText Transfer Protocol (HTTP) are belong to this layer [6]. This layer ensures that data collected and processed can be obtained or viewed by the user. Transport layer is responsible to transport data from end to end. As the sensors collect data from the surrounding environment, in order for it to transport the collected data to another layer for further processing, transport layer is needed. Transport layer is responsible for building a connection with the target server and ensure the process of transferring data from client to server is successful. High reliability and security of transport layer is very much needed in IoT environment to ensure huge amount of data can be transferred from one place to another without having issues with slow connection and data integrity breach while transporting the data. The process layer is the fundamental part of any IoT device as it is responsible to convert the data into information that can be acknowledged by the user or machine. A huge amount of data streams is expected to be received from the transport layer. Thus, the process layer will apply the required analysis technique and store the data to be used later [7]. The function of processing layer is to analyze the captured data and to store the data inside the cloud. Edge computing is one of the applications in the processing layer. This application analyzes the data received before transferring it to the cloud storage to be stored.

According to [7], business layer is responsible to manage the whole IoT system, business module and profit. The business model is sitting at the top layer of the five layers of IoT architecture as this layer is responsible to control all the lower layer in the architecture. As the IoT technology is increasing in demand, there is no doubt that a business entity is required to provide the services of this technology to user or company. Therefore, all the layer in the architecture must be governed or provided by the provider company.

## 2.3 Types of Encryption Security Available

Encryption means encoding information to prevent unauthorized parties from viewing the information. Encryption required a set of key that can only be known by the sender and the receiver to decrypt the information. Encryption is one of the security methods commonly used to secure a connection on the internet. Since IoT required internet to provide its services, encryption method can be implemented to protect the IoT system.

Currently, there is no set of encryption security that being created specifically to secure the IoT system. According to [8], even though there are available encryption protocol to protect the security of the internet, but none of them is designed specifically to work in IoT environment. Nevertheless, three of the protocols listed below are commonly used by

researchers in order to provide encryption type of security for IoT environment. SSL/TLS, Internet Protocol Security (IPSec) and Quick UDP Internet Connection (QUIC) protocols are used to provide security to data being transferred in by encrypt it [9–11]. The next three subsections below explained the details about all three security protocols.

### 2.3.1 SSL/TLS

SSL/TLS protocol is used to create a secure session between a host and a server in the localized protected local area network [12]. SSL/TLS is a cryptographic protocol that provide end to end security of data transfer between applications over the internet. It is worth mentioning that SSL/TLS only secured the process of transferring the data and not the end system. The motivation behind SSL/TLS is to protect the communication channel from eavesdropped or alteration of the content. This protocol suitable for IoT as one of the potential threats faced by IoT is eavesdropping [4, 5]. SSL/TLS encryption are designed to work with Transmission Control Protocol (TCP) [11].

Prantl et al. [13] investigated the effect of securing MQTT protocol with TLS. The influence of TLS on the throughput, energy harvesting and connection establishment time was analyzed. Authors found that implementing TLS has a considerable negative impact on the system performance. For high packet loss settings, TLS can negatively influence the throughput, establishment time and energy consumption. However, with small packet loss settings, TLS has no impact on throughput and energy efficiency. Moreover, authors conclude that there is no relation between throughput and energy efficiency. The study conducted for single client only. In [14], the influence of implementing TLS on energy consumptions was evaluated. It was found that the consumption is increased by 6.7% for high packet loss settings. The opposite was reported in [15]. Authors conclude that energy consumption increased as QoS level increased. QoS 2 consumes almost 10 mW than QoS 0. The effect of TLS in network performance in terms of throughput and latency has been evaluated by Laaroussi and Novo [16]. The secured MQTT showed less performance than unsecured MQTT. The throughput and the latency have negatively influenced by implementing TLS. Testing secured MQTT in android environment using TLS is presented in [17]. The security protocol is found to increase the connection time. An automated tool that can help the developers of MQTT protocol to identify the vulnerabilities related to SST/TLS was presented in [18].

The process of asking for encryption session began with client sends a hello message to the server asking for an encrypted session. Hello message from a client contains the information regarding the set of cipher suite it has to in order to provide an encryption session with a server. The server would respond with its choice of cipher suite to be used for this encryption session and a public key to be used by the client to generate ‘pre-master’ key. The received public key would be used by the client to encrypt the message sent to the server. Once the server has received the encrypted message from the client, it would use its private key to decipher the message. The reason a server has a couple of public and private key is to ensure only the server able to decrypt the encrypted message sent by the client using its private key.

After manage to decrypt the message sent by the client, a server would send a ‘pre-master key to the client. This pre master key would be used by the client and the server to determine a shared secret key for both. The secret shared key is the one would be used by both for their encryption communication. It required at least 2-RTT (Round

Trip Time) for TLS to complete the negotiation process [19]. Figure 1 shows the overview of the handshake process.

### 2.3.2 QUIC

QUIC is a transport protocol that works on top of User Datagram Protocol (UDP) to provide a fast, reliable and encrypted services to internet connection. The motivation behind the creation of QUIC is to provide faster connection for internet and to provide an alternative in transport protocol. According to [20], QUIC is proposed to minimize the latencies of both connection and transport predictable in the prevailing TCP. QUIC sits in the middle of UDP and HTTP/2. Therefore, QUIC is not just part of transport protocol but also part of the application protocol. One of the abilities of QUIC is to provide an encryption security. To provide security for the data transferred, QUIC proposes an encryption method almost similar like TLS [21]. The encryption method is using secret shared key to encrypted data transferred. Different with TLS which required at least 2-RTT in negotiation process [19, 20] when used with TCP connection, QUIC requires 1-RTT for key exchange process to occur. This 1-RTT process is achieved by QUIC thanks to its build in security system during its negotiation process [19]. This process reduces the latency of negotiation between server and client.

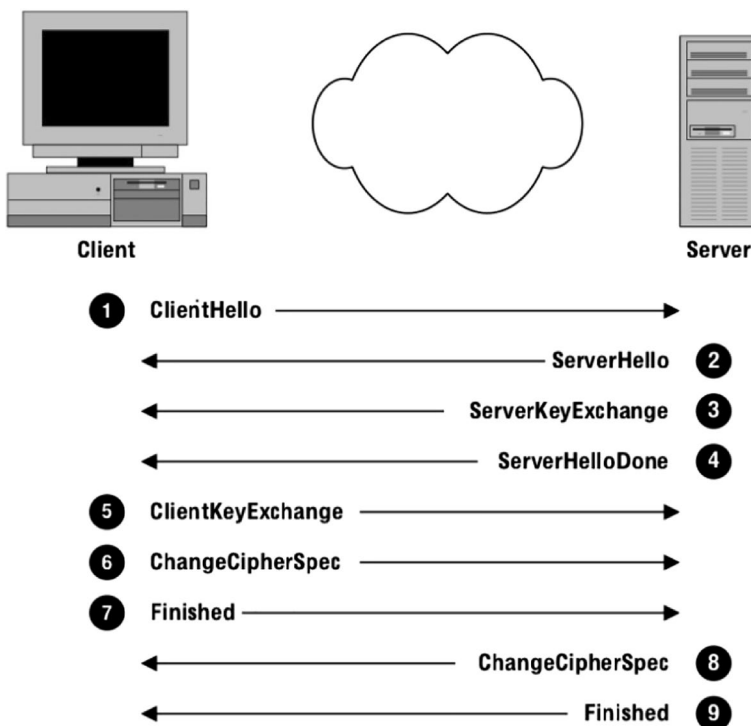


Fig. 1 Process of secret key exchange [11]

### 2.3.3 IPSEC

IPSec defines a set of protocols for securing IP communication. Differ from TLS and DTLS which offer protection at transport layer, the IPSec offer a protection within the IP layer. IPSec offers 2-mode protection as shown in Fig. 2. First mode is Authentication Header (AH) and second mode is Encapsulating Security Payload (ESP). AH mode for IPSec provides source authentication, data integrity but not privacy. AH provides an encryption at the header and the payload of the IP layer. The data integrity is obtained by using message digest which is generated using certain type of algorithm. On the other hand, source authentication is obtained by sharing a secret key to create message digest. The important of message digest is that it provides protection from any alteration or changes to any part of the message. ESP provides data confidentiality and authentication. ESP provides protection mechanism to the IP Datagram portion of the IP packet. Both ESP and AH use the same algorithm but the difference is the coverage of the protection. The AH offer protection to from the header to all portion of the IP packet but ESP only provide protection to the IP Datagram [10].

## 2.4 Internet of Things Application Layer Protocols

### 2.4.1 MQTT

MQTT works on publish/subscribe principle. MQTT client is enabled to subscribe to a topic or publish to topic in broker. Basically, MQTT client is also able to do both functions, subscribing and publishing [22–24]. MQTT client is a telemetry device that gathers data from a surrounding environment and publish it to the broker when it is in publishing mode. If a user subscribes to a topic, it will receive the data based on the topic it subscribes to. A server in MQTT is known as a broker. A broker would store a topic published by publisher and publish that topic to the subscriber that subscribe to that topic. A message from a client would be sent to a broker and a broker later would distribute the message to any clients that subscribe to the topic. This mechanism make this messaging protocol a message-oriented protocol [25]. MQTT protocol is common in IoT system and it is located at the application layer. Lightweight and able to ensure reliability of the transport data makes

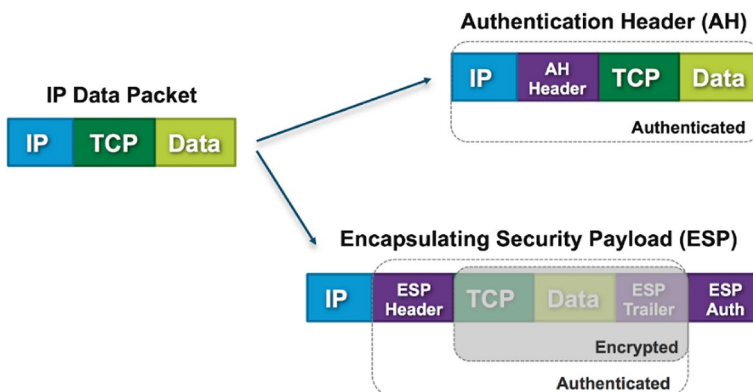


Fig. 2 Difference between AH and ESP [10]

this protocol a common thing in IoT environment. MQTT network bandwidth and device resource requirements make it suitable for Machine to Machine (M2M) connection which is part of the IoT [21, 22]. To ensure the reliability of this protocol, it required a TCP connection to deliver the message.

Quality of Service (QoS) level in MQTT is the assurance level that a message would be delivered from a client to a broker or vice versa. A client can have a different QoS levels when publishing to a broker and a broker could have a different QoS levels when publishing to subscriber. Both broker and client could have different preference levels of QoS and still able to operate in the same environment. There are three QoS levels available in MQTT. The first level is “0”. This level is to ensure the delivery of the message at most once. The sender would just send a message to the destination and would not consider anything that happens to the message after the transmission. The sender would not receive any acknowledgement that a receiver has received the message, nor a receiver would send any acknowledgment when it receives the message. This level reduces the overhead making it suitable for a client that have low bandwidth. The second level of QoS is “1”. This level ensures the delivery of the message at least once. The sender would wait for the acknowledgement message from the receiver after it has delivered the message. If the sender does not receive the acknowledgement message in certain amount of time, it will send another copy of message to the subscriber. This process continues until it receives the acknowledgement message from a subscriber. This process encourages a duplication of the message to be sent multiple times to the receiver. Therefore, a receiver needs a logic to differentiate between the original message and duplicate message to avoid processing it multiple times. The third level of QoS of MQTT protocol is “2”. This level ensures that the delivery of the message is exactly one. This level required 2 flows between a server and client. A message published with QoS level 2 is considered successfully delivered once after the sender confirms that it has been successfully received once. This level required the largest overhead compared to the other two levels [26].

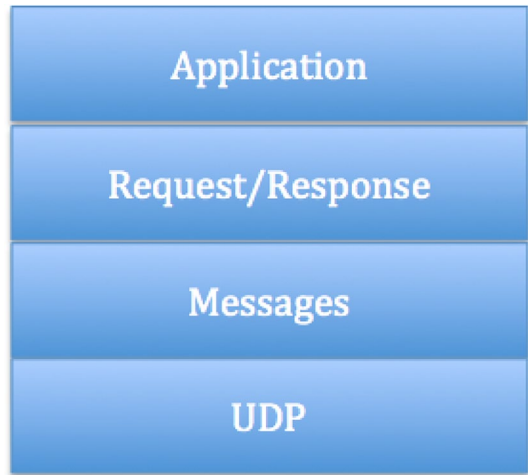
## 2.4.2 Constraint Application Protocol (CoAP)

CoAP is a protocol developed by Internet Engineering Task Force (IETF) for an application layer to support smart devices to be connected to the internet. This protocol is designed to have similar features to HTTP but in a lightweight version. It is designed based on Representation State Transfer (REST) architecture which is a general design for accessing internet resources. CoAP works with UDP transport protocol. It is designed to be used at low-power and lossy networks. Figure 3 below shows the CoAP two-layer structure model [12]. The first layer structure is message and UDP. The message layer of CoAP is designed to work with UDP protocol. The second layer is request/response layer that deals with the request and response of the transferred message.

## 2.4.3 REST

REST is an Application Program Interface (API). A system that follows REST based API, is known as RESTful API. An important part of REST is its Uniform Resource Identifiers (URI). URI is a string character used to identify a resource on the internet either by location or by name or both. A URI is used to access the data. A URI can retrieve an information using different protocols. The most common protocol used is HTTP. A REST API uses

**Fig. 3** CoAP Layered structure model [12]



similar uniform interface operation as HTTP. These operations are (PUT, GET, POST and DELETE). For a REST to work properly, it needs to use all of these operations [27].

#### 2.4.4 Hypertext Transfer Protocol (HTTP)

HTTP is a protocol proposed for an application-level for distribution, collaborative and hypermedia information system. It is the protocol that mainly used on to communicate with a web server to retrieve the information in it and display the information in the web browser of the customer. HTTP uses reliable-data transmission such as TCP and it guarantees that your data will not be damaged or lost during transmission. For HTTP to work, it needs a HTTP server and HTTP clients. On the internet, a data lived inside a server. A client sends a HTTP request to a server and a server return the HTTP response to the clients [28].

### 3 Methodology and Testbed Design

To investigate the feasibility of using SSL/TLS encryption within the IoT domain, and to quantify its effect on common IoT devices, the following experimental set up is used for the implementation and testing.

#### 3.1 Hardware/Software Configuration

To implement this testbed, the private network of MQTT should be constructed on the first hand. The hardware used to implement MQTT broker is Raspberry Pi 4 B + micro-processor. Mosquitto broker is installed inside Raspberry Pi to transform it into MQTT broker. For MQTT client, Wi-Fi enabled ESP 32 microcontroller with a light sensor is used. Lithium battery will be used to power ESP 32 microcontroller while the Pi is powered using the main supply. Wireshark software is used to sniff and dissect packets from the transaction between MQTT client residing on the ESP 32 device and the broker residing on the Pi to monitor the encrypted and unencrypted packet transfer and to



calculate the traffic encryption overhead. Open SSL is used to implement the encryption on both ends and to create and maintain the TSL/SSL server certificate into MQTT server and clients.

### 3.2 Testbed Physical Design Structure

The design structure is shown in Fig. 4. MQTT Broker is the server of the MQTT. The broker would subscribe to a certain topic so that the messages that is being published by the client would be received and stored. The messages published mainly the process data that it received from its surrounding. The process of publishing the data can be continuous and therefore there may be thousands of messages being published after certain times. Internet is used to transport the data from the clients to the server and vice versa. For this research, a Wi-Fi internet connection to provide connection for ESP 32 and MQTT broker was used.

### 3.3 Methodology

To benchmark and quantify the overhead and consumed energy, the base line procedure of stress-testing the testbed without the use of TLS connection is performed first. The procedure is then repeated using an established TLS connection. The performance Metrics used are the generated overhead (in bytes) and the consumed energy (in Joules) only. All the QoS levels supported by MQTT broker are tested. The steps below demonstrate the procedures used.

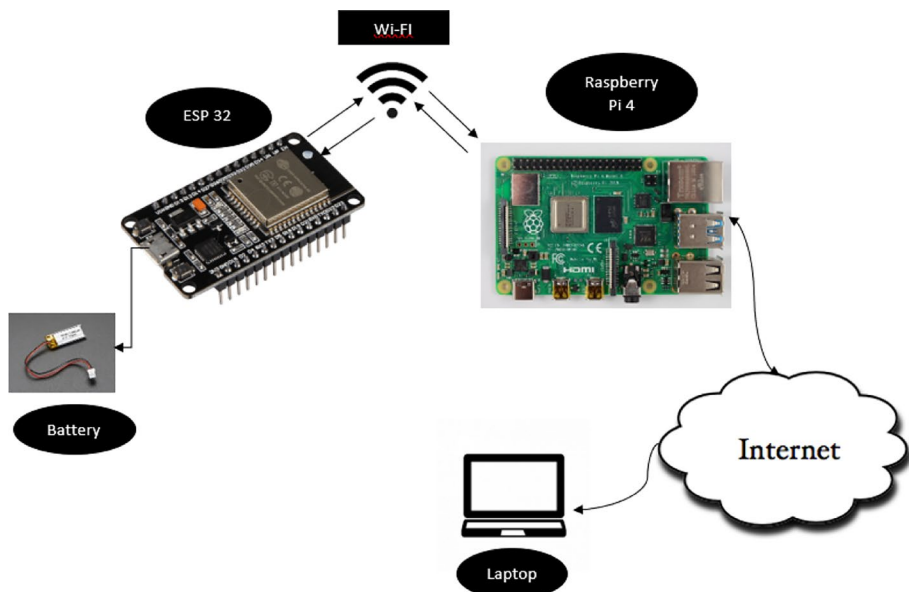


Fig. 4 Testbed design structure

#### 4 Procedure without TLS Connection:

1. Go to raspberry pi command window.
2. Type “mosquito\_sub -p 1883 -t InTopic” for server to subscribe to the topic “InTopic”.
3. Go to Arduino software, write the code into the ESP 32 board
4. Set the QOS level for ESP 32.
5. Increase the number of messages transmitted by ESP 32 every 1 min.
6. In the meantime, open Wireshark application and start packet sniffing while the client (ESP 32) publish message to “InTopic”.
7. Stop the Wireshark packet sniffing after every number of messages transmitted and record the number of bytes and number of bit rate for each transmission.
8. Use the energy formula to calculate the energy of the ESP 32.
9. Repeat procedure 3 until 6 by changing the number of QOS level of the Arduino ESP 32.

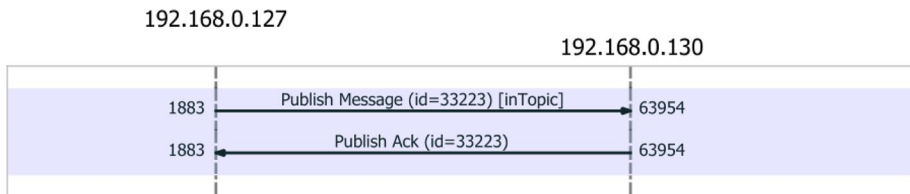
#### 5 Procedure with TLS Connection

1. Go to raspberry pi command window
2. Type “mosquito\_sub -p 8883 -t InTopic -h raspberrypi --cafile /etc/mosquito/certs/ca3.pem” for server to subscribe to the topic “InTopic”.
3. Go to Arduino software, write the code into the ESP 32 board.
4. Set the QOS level for esp 32.
5. Increase the number of messages transmitted by ESP 32 every 1 min
6. In the meantime, open Wireshark application and start packet sniffing while the client (ESP 32) publish message to “InTopic”.
7. Stop the Wireshark packet sniffing after every number of messages transmitted and record the number of bytes and number of bit rate for each transmission.
8. Use the energy formula to calculate the energy of the ESP 32.
9. Repeat procedure 3 until 6 by changing the number of QOS level of the Arduino ESP 32.
10. Plot graph of energy consumption of ESP 32 and number of bytes for every QOS level using for both with TLS and without TLS data.

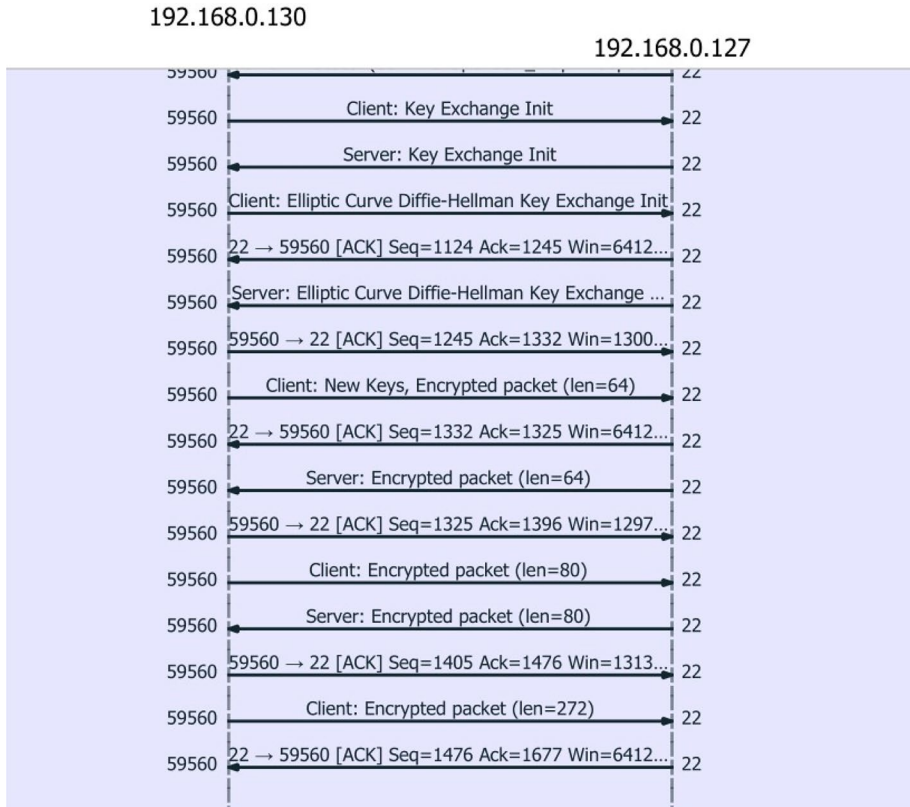
#### 5.1 Message Exchange

In order to calculate the energy consumption of ESP 32, Eqs. (1), (2), and (3) below are used. During transmission, ESP 32 requires on average 0.615 milliwatt of power to transmit data and 255 milliwatt to receive data [29]. This is the energy required to operate the electronic circuits and components on the board. The bytes received and transmitted during the exchange of the data between ESP 32 and MQTT broker are collected in order to calculate the amount of energy consumed during the transmission and receiving for ESP 32. The consumed energy by the ESP 32 device is calculated as follows.

$$T_x(\text{Joules}) = 0.615 \frac{\text{Joules}}{\text{Seconds}} \times \frac{\text{Seconds}}{\text{Bytes}} \times \text{Bytes} \quad (1)$$



**Fig. 5** Message exchange between ESP 32 and MQTT broker without TLS encryption



**Fig. 6** Message exchange between ESP 32 and MQTT broker with TLS encryption

$$R_x(\text{Joules}) = 0.255 \frac{\text{Joules}}{\text{Seconds}} \times \frac{\text{Seconds}}{\text{Bytes}} \times \text{Bytes} \quad (2)$$

$$\text{Total Energy Consumed}(\text{Joules}) = T_x(\text{Joules}) + R_x(\text{Joules}) \quad (3)$$

To appreciate system complexity and the amount of extra overhead exchanged during the invoking of the TLS connection, Figs. 5 and 6 are created to demonstrate the time-event diagram of the message exchange taking place between the MQTT Broker and the client without TLS and with TLS encryption, respectively.

During the message exchange for MQTT broker and Client without TLS encryption, a client needed to subscribe to the topic that broker have before able to publish messages in the topic. After clients have subscribe to the topic available in the server, messages can be published into the subscribe topic. After done publishing the message, packet of acknowledgement is sent to the client by the server indicating that the message is received. The connection is simple and does not contain any handshake prior. Figure 5 shows the screenshot of the packet exchange of one message during the experiment.

During the transmission of messages for TLS connection, a client and server need to perform a key exchange mechanism before message can be transmitted. During the mechanism, both client and server need to agree on the type of key that would be used during transmission. After the key have been agreed, server would send its public key to client and client would use the public key to encrypt the message it is about to send. After server have received the publish message, it would then send an acknowledgment packet to the client. This process is complex compared to connection without TLS. Figure 6 shows the encryption process during transmission of one message during TLS encryption connection.

From both figures shown above, it can be seen the additional complexity required for TLS connection need to be established in order for ESP 32 client to publish message to MQTT broker. TLS encryption connection between ESP 32 and MQTT broker required more packet exchange during the transmission and thus increase the number of overhead bytes during transmission of messages. ESP 32 IP address is 192.168.0.127 and MQTT Broker IP address is 192.168.0.130.

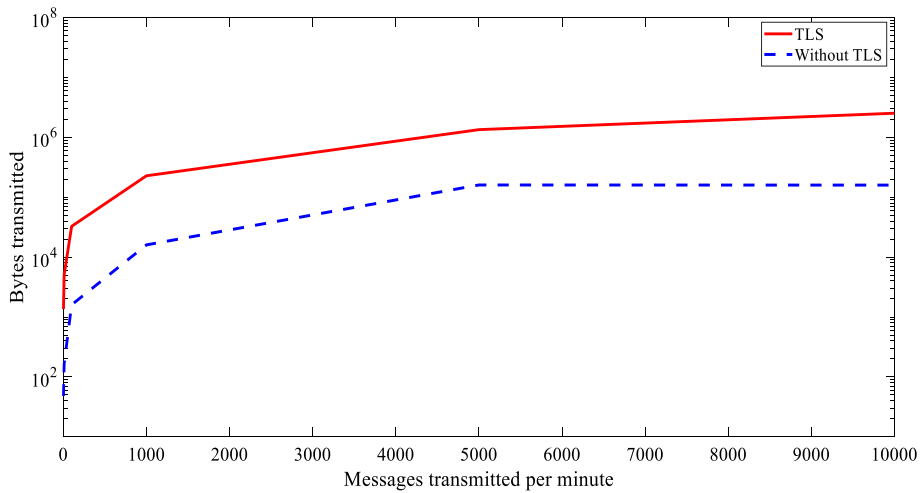
## 6 Results and Discussion

### 6.1 System Overhead

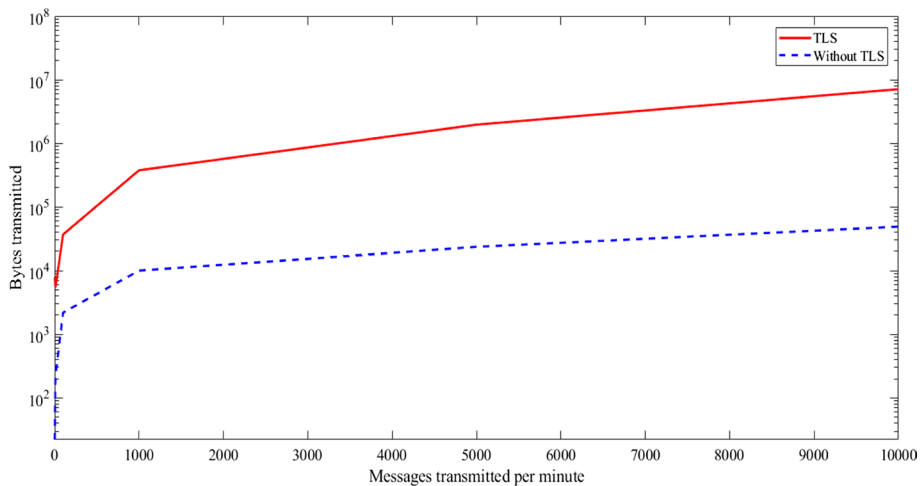
System overhead is determined based on the different number of bytes between TLS and without TLS connection. The system overhead is predicted to be large for TLS connection as more bytes is required in order to establish the connection between clients and broker. This prediction is made based on the message exchange explained above as the complexity of the message exchange increase for TLS connection compared to without TLS connection. Figures 7, 8 and 9 below show the comparison of number of bytes transmitted during the whole process of subscribing and publishing of ESP 32 with MQTT Broker for different QOS levels with different number of messages transmitted per minute for TLS and without TLS encryption. It can be seen from all three figures that bytes transmitted without TLS is much lower compared to bytes transmitted with TLS connection. The large overhead is due to the complexity occurred during the message exchange as key exchange taking places between server and client and therefore produce large number of bytes during the process. This overhead occurred for every QOS level. The QOS 2 has the biggest overhead compared to other QOS levels. This is due to QOS 2 mechanism which required a client to publish message exactly once.

### 6.2 Energy Consumption

Based on the formula used to calculate the total energy consumption for ESP 32, it can be found that increasing in number of bytes transmitted would lead to increasing of energy consumed. Figures 10, 11 and 12 show the energy consumption for different



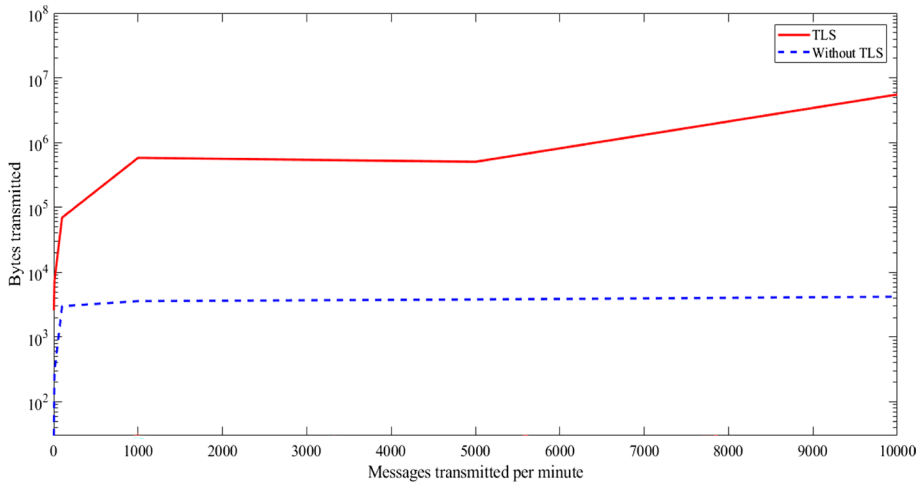
**Fig. 7** Number of bytes transmitted for QOS 0



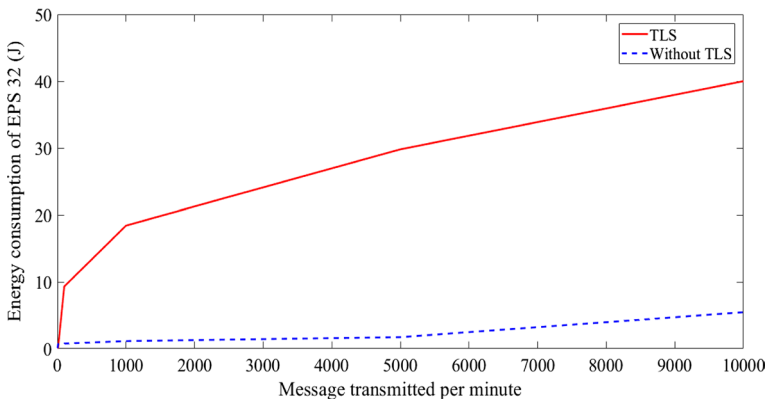
**Fig. 8** Number of bytes transmitted for QOS 1

levels of QOS. From the graphs, energy can be concluded for every QOS level, energy consumption for ESP 32 is huge compared to energy consumption of ESP 32 without TLS encryption. At QOS 1, TLS encryption consume more than 10 times the energy of the ESP 32 board compared to without TLS encryption. For TLS connection of QOS 2, the energy consumed is the highest compared to others TLS connection. QOS 2 consumed more than 70 J of energy for it to transmit 10,000 messages per minute.

From the figures below, one can deduct that without TLS invocation, unsecured MQTT connection with QOS 2 consume the highest level of energy followed by QOS



**Fig. 9** number of bytes transmitted for QoS 2

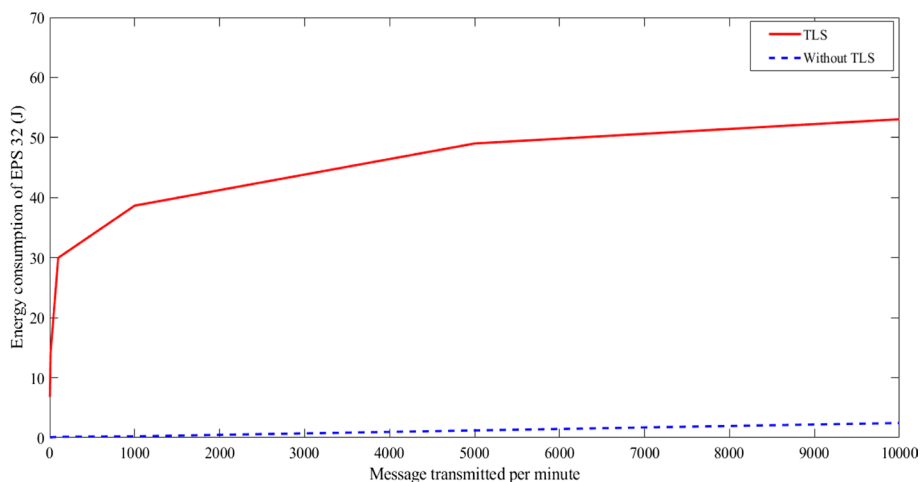


**Fig. 10** Energy consumption of ESP 32 with QoS 0

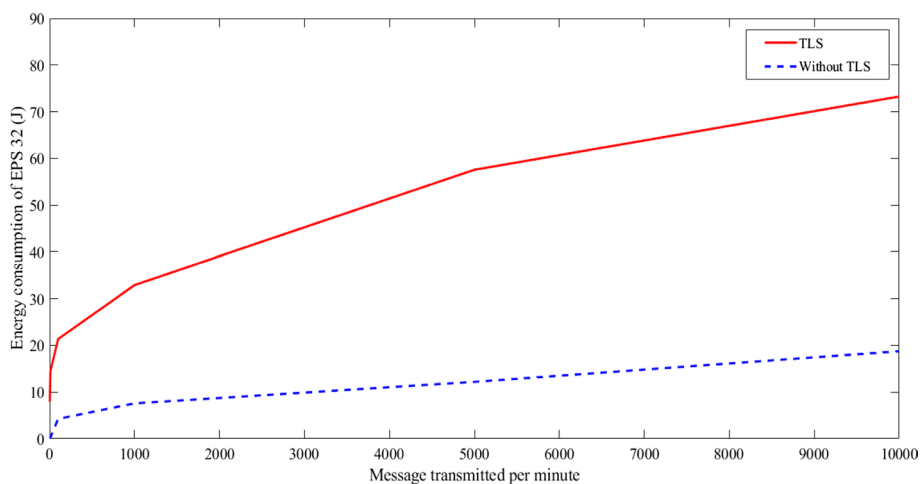
1 level in comparison to QoS 0. This is true regardless of what number of messages is offered to the ESP 32 board.

### 6.3 Memory Storage Consumption and Execution Time

This subsection discusses the complexity levels imposed due to the invocation of TLS session on top of the MQTT message exchange. It is translated in terms of ratio of increase in consumed time, time consumed without TLS, and time consumed with TLS for the three QoS levels, as shown in Tables 1, 2, and 3, respectively. Before we delve into observing the results for the execution time, it is worth mentioning the following; Since different versions of ESP 32 hardware exists with different featured RAM sizes, speed of processors utilized, and different Wi-Fi modems implemented on the board, let alone comparing different IoT



**Fig. 11** Energy consumption of ESP 32 with QOS 1



**Fig. 12** Energy Consumption of ESP 32 with QOS 2

**Table 1** Level QOS 0 performance metrics with/without TLS connection

| Number of messages per minute | Time consumed without TLS (micro second) | Time consumed with TLS (micro second) | Ratio of increase in consumed time |
|-------------------------------|------------------------------------------|---------------------------------------|------------------------------------|
| 2                             | 0.2441                                   | 6.9173                                | 28.3333                            |
| 10                            | 0.8341                                   | 25.9094                               | 31.0610                            |
| 100                           | 8.1380                                   | 166.1784                              | 20.4200                            |
| 1000                          | 81.4006                                  | 1158.8540                             | 14.2364                            |
| 5000                          | 814.1225                                 | 6828.6950                             | 8.3878                             |
| 10,000                        | 810.0739                                 | 12,837.8900                           | 15.8478                            |

**Table 2** Level QoS 1 Performance metrics with/without TLS connection

| Number of messages per minute | Time consumed without TLS (micro second) | Time consumed with TLS (micro second) | Ratio of increase in consumed time |
|-------------------------------|------------------------------------------|---------------------------------------|------------------------------------|
| 2                             | 0.111898                                 | 39.87122                              | 356.318182                         |
| 10                            | 1.118978                                 | 27.75065                              | 24.8                               |
| 100                           | 11.18978                                 | 187.2559                              | 16.7345455                         |
| 1000                          | 51.13729                                 | 1913.574                              | 37.4203302                         |
| 5000                          | 119.6187                                 | 9987.752                              | 83.4965558                         |
| 10,000                        | 248.3012                                 | 36,132.43                             | 145.518538                         |

**Table 3** Level QoS 2 performance metrics with/without TLS connection

| Number of messages per minute | Time consumed without TLS (micro second) | Time consumed with TLS (micro second) | Ratio of increase in consumed time |
|-------------------------------|------------------------------------------|---------------------------------------|------------------------------------|
| 2                             | 0.152588                                 | 13.18359                              | 86.4                               |
| 10                            | 1.525879                                 | 38.57422                              | 25.28                              |
| 100                           | 15.25879                                 | 352.5391                              | 23.104                             |
| 1000                          | 183.1055                                 | 2954.183                              | 16.1337778                         |
| 5000                          | 193.7866                                 | 2566.081                              | 13.2417848                         |
| 10,000                        | 213.623                                  | 28,136.47                             | 131.710857                         |

gear from different manufacturers using different capillary technologies (e.g., ZigBee, LoRa, LTE, ...etc.), it is difficult to quantify the amount of time consumed by different versions of the same IoT device sporting different radio technologies. Hence, the presented results attempt to capture the trendline behavior instead of the actual execution time consumed, even though it is also presented for appreciation purposes only.

From Table 1 for QoS 0 level, the time consumed by invoking TLS on top of MQTT to the case of exchanging MQTT messages without the TLS, increases with the increase of the number of transmitted messages per minute. This is expected, as the number of transmitted messages increases, the generated overhead increases as shown in Figs. 7, 8 and 9. Notice that since IoT devices differ tremendously in their computational capabilities and memory resources, the Tables 1, 2, and 3 do also present the trend lines of the execution times consumed as a ratio of increase (e.g., indicating the number of folds in increase). It is also noteworthy that it is cumbersome to extract the actual execution time from other processing delays invoked due to bad channel retransmissions, propagation time, actual message transmission rate, buffering delay, ...etc. Hence, these results should only be interpreted as an indication of the expected behavioral trend with appreciable upper and lower bounds rather than the actual exact execution time on the processor, as design engineers expect.

It is also worth noting that as the session is started and the TLS is invoked, at the beginning, the session setup control messages exchange is usually high as shown in Fig. 6. Hence, the notch noticed at the rate of 2 messages per minute. Afterwards, the ration of consumed time decreases as expected. This is true for all the QoS levels and throughout the repeated experiments, even though it is somewhat more emphasized for the case of QoS 1 results presented in Table 2. Furthermore, the above discussed results are evidently confirmed by the results for the consumed energy levels in Joules. At the starting message



rate, it was only an increase of 2 times whereas it jumped to 16 folds as the messaging rate reached 10,000 messages per minute. The trend illustrating the complexity level demonstrated by the TLS invocation on top of MQTT message exchange.

In conclusion, the above discussed results illustrate further the linearly increasing amount of memory storage needed to cope with the stress- testing on the ESP 32 modem for the different QoS modes of MQTT server operation.

Overall comparison between connection of ESP 32 and MQTT broker in terms of energy consumption and bytes transmitted, one can conclude that connection with TLS encryption produce a large overhead and large energy consumption compared to connection without TLS encryption. Based on the result obtained, a conclusion can be made that TLS encryption may not be suitable for the IoT environment after all. This is due to the lack of energy storage, small processing capabilities and small capacity in data storage of IoT products. The nature of IoT which required it to transmit data every second would be affected by the implementation of TLS in the system. This motivates pushing the TLS implementation to the edge Gateway instead of the individual IoT nodes, as it is the trend nowadays.

## 7 Future Works

Since security is still is the main concern in IOT environment, therefore a suggested future works is to implement other types of security available in order to find the best kind of security to protect this fragile network. In this paper, it has proved the lack of energy efficient, high in system complexity and high in data overhead when TLS is implemented inside MQTT environment. Future work should focus on how to use other type of security to work better in IOT environment. The criterion that needs to be meet for that security are low in power consumption, low in system complexity and low in system overhead.

## 8 Conclusions

The objectives of this paper are to implement and analyze the performance of TLS encryption in MQTT environment. Based on the result obtained above, it can be concluded that TLS encryption security is not suitable for IoT environment due to its high energy consumption, high system complexity and high overhead. For small devices like IoT where power, processing capabilities and storage capacity are limited, it is not practical and efficient to provide security to IoT using TLS encryption. Therefore, other security protocols that are low in power consumption and low in system overhead is required.

**Funding** This work was conducted in IoT and Wireless Communication Protocols Lab, ECE Department, International Islamic University Malaysia (IIUM).

**Data Availability** This research has no associated data to it that is relevant to the content of this article.

## Declarations

**Conflict of interest** The authors have no conflicts of interest to declare that are relevant to the content of this article.

## References

1. Canedo, J., & Skjellum, A. (2016). Using machine learning to secure IoT systems. In *2016 14th annual conference on privacy, security and trust (PST)*, IEEE. pp. 219–222. <https://doi.org/10.1109/PST.2016.7906930>.
2. Statista. Number of IoT connected devices worldwide 2019–2021, with forecasts to 2030, 22 November, 2022. <https://www.statista.com/statistics/1183457/iot-connected-devices-worldwide/>.
3. Kasinathan, P., Pastrone, C., Spirito, M. A., & Vinkovits, M. (2013). Denial-of-service detection in 6LoWPAN based internet of things. In *2013 IEEE 9th international conference on wireless and mobile computing, networking and communications (WiMob)*, IEEE. pp. 600–607. <https://doi.org/10.1109/WiMOB.2013.6673419>.
4. Ali, I., Sabir, S., & Ullah, Z. (2016). Internet of things security, device authentication and access control: a review. *International Journal of Computer Science and Information Security IJCSIS*, 14(8), 456–466.
5. Dineva, K. & Atanasova, T. (2019). Security in iot systems. In *19th international multidisciplinary scientific geoconference SGEM 2019*, pp. 569–578, no 2.1. <https://doi.org/10.5593/sgem2019/2.1/S07.075>.
6. Tandale, U., Momin, B. & Seetharam, D. P. (2017). An empirical study of application layer protocols for IoT. In *2017 International conference on energy, communication, data analytics and soft computing*, pp. 2447–2451. <https://doi.org/10.1109/ICECDS.2017.8389890>.
7. Sethi, P., & Sarangi, S. R. (2017). Internet of things: architectures, protocols, and applications. *Journal of Electrical and Computer Engineering*.
8. Tiburski, R. T., Amaral, L. A., De Matos, E., De Azevedo, D. F., & Hessel, F. (2016). The role of lightweight approaches towards the standardization of a security architecture for IoT middleware systems. *IEEE Communications Magazine*, 54(12), 56–62. <https://doi.org/10.1109/MCOM.2016.1600462CM>
9. Jokela, P., Moskowitz, R., & Nikander, P. (2008). Using the encapsulating security payload (ESP) transport format with the host identity protocol (HIP). *RFC5202*.
10. Bensalah, F., El Kamoun, N., & Bahnasse, A. (2017). Evaluation of tunnel layer impact on VOIP performances (IP-MPLS-MPLS VPN-MPLS VPN IPsec). *International Journal of Computer Science and Network Security (IJCSNS)*, 17(3), 87.
11. Thomas, S. (2000). *SSL and TLS essentials, securing the web* (p. 3). John Wiley and Sons.
12. Chen, X. (2014). Constrained application protocol for internet of things. URL: <http://www.cse.wustl.edu/~jain/cse574-14/ftp/coap>
13. Prantl, T., Iffländer, L., Herrnleben, S., Engel, S., Kounev, S., & Krupitzer, C. (2021). Performance impact analysis of securing mqtt using tls. In *Proceedings of the ACM/SPEC international conference on performance engineering*, pp. 241–248.
14. Baranauskas, E., Toldinas, J., & Lozinskis, B. (2019). Evaluation of the impact on energy consumption of MQTT protocol over TLS. In *CEUR workshop proceedings: IVUS 2019 international conference on information technologies: Proceedings of the international conference on information technologies*, Kaunas, Lithuania, April 25, 2019, Vol. 2470, pp. 56–60. CEUR-WS.
15. Shapsough, S., Aloul, F., & Zualkernan, I. A. (2018). Securing low-resource edge devices for IoT systems. In *2018 International symposium in sensing and instrumentation in IoT Era (ISSI)*, IEEE. pp. 1–4.
16. Laaroussi, Z., & Novo, O. (2021). A performance analysis of the security communication in CoAP and MQTT. In *2021 IEEE 18th Annual consumer communications & networking conference (CCNC)*, IEEE. pp. 1–6.
17. Silva, C., Toasa, R., Martinez, H. D., Veloz, J., & Gallardo, C. (2017). Secure push notification service based on MQTT protocol for mobile platforms. In *XII Jornadas Iberoamericanas de Ingeniería de Software e Ingeniería del Conocimiento y Congreso Ecuatoriano en Ingeniería de Software*, pp. 69–84.

18. Alghamdi, K., Alqazzaz, A., Liu, A., & Ming, H. (2018). Iotverif: An automated tool to verify ssl/tls certificate validation in android mqtt client applications. In *Proceedings of the eighth ACM conference on data and application security and privacy*, pp. 95–102.
19. Saverimoutou, A., Mathieu, B., & Vaton, S. (2017). Which secure transport protocol for a reliable HTTP/2-based web service: TLS or QUIC?. In *2017 IEEE symposium on computers and communications (ISCC)*, IEEE. pp. 879–884. <https://doi.org/10.1109/ISCC.2017.8024637>.
20. Seufert, M., Schatz, R., Wehner, N., Gardlo, B., & Casas, P. (2019). Is QUIC becoming the new TCP? On the potential impact of a new protocol on networked multimedia QoE. In *2019 Eleventh international conference on quality of multimedia experience (QoMEX)*, IEEE. pp. 1–6. <https://doi.org/10.1109/QoMEX.2019.8743223>.
21. Lampkin, V., Leong, W. T., Olivera, L., Rawat, S., Subrahmanyam, N., Xiang, R., & Locke, D. (2012). *Building smarter planet solutions with mqtt and ibm websphere mq telemetry*. IBM Redbooks.
22. Wukkadada, B., Wankhede, K., Nambiar, R., & Nair, A. (2018). Comparison with HTTP and MQTT in internet of things (IoT). In *2018 International conference on inventive research in computing applications (ICIRCA)*, IEEE. pp. 249–253. <https://doi.org/10.1109/ICIRCA.2018.8597401>.
23. Habaebi, M. H., Al-Haddad, A., Zyoud, A., & Hijazi, G. (2018). Micro search engine for IoT: An IoT search engine prototype for private networks. *Recent Advances in Electrical and Electronic Engineering, 11*(2), 123–131. <https://doi.org/10.2174/2352096511666180117144450>
24. Hijazi, G., Hadi Habaebi, M., Al-Haddad, A., & Zyoud, A. M. (2021). Stress testing MQTT server for private IOT networks. *International Journal of Electronics and Telecommunications, 67*(2), 229–234.
25. Carlsson, F., & Eriksson, K.-G. (2018). *Comparison of security level and current consumption of security implementations for MQTT* (Master Dissertation). Retrieved from <http://urn.kb.se/resolve?urn=urn:nbn:se:hj:diva-40760>.
26. Yassein, M. B., Shatnawi, M. Q., Aljwarneh, S., & Al-Hatmi, R. (2017). Internet of things: Survey and open issues of MQTT protocol. In *2017 International conference on engineering & MIS (ICEMIS)*, IEEE. pp. 1–6. <https://doi.org/10.1109/ICEMIS.2017.8273112>.
27. Rodríguez, C., Baez, M., Daniel, F., Casati, F., Trabucco, J. C., Canali, L., & Percannella, G. (2016). REST APIs: A large-scale analysis of compliance with principles and best practices. In *International conference on web engineering*, Springer, Cham. pp. 21–39. [https://doi.org/10.1007/978-3-319-38791-8\\_2](https://doi.org/10.1007/978-3-319-38791-8_2).
28. Fielding, R., Gettys, J., Mogul, J., Frystyk, H., Masinter, L., Leach, P., & Berners-Lee, T. (1999). Hypertext transfer protocol–HTTP/1.1.
29. ESP 32 Series Datasheet (2018). [https://www.espressif.com/sites/default/files/documentation/esp32\\_datasheet\\_en.pdf](https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf)

**Publisher's Note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



**Iqbal Luqman Bin Mohd Paris** received his bachelor in Communication Engineering from International Islamic University Malaysia (IIUM) in 2020. His research interest includes computer networking, IoT and security.



**Mohamed Hadi Habaebi** (M'99–SM'16) received his first degree from Civil Aviation and Meteorology High Institute, Esbeiaa, Libya, in 1991, his M.Sc. degree in electrical engineering from Universiti Teknologi Malaysia in 1994, and his Ph.D. degree in computer and communication systems engineering from Universiti Putra Malaysia in 2001. He is the former Head of the Department and a professor at the Electrical and Computer Engineering, International Islamic University Malaysia, Kuala Lumpur, Malaysia. He is the founding member of IoT and wireless Communication Protocols Laboratory with the same department. He has supervised many Ph.D. and M.Sc. students, authored or co-authored more than 200 articles and papers, and sits on the editorial board of many international journals. He is actively reviewing and publishing in computer communications, wireless sensor and actuator networks, cognitive radio, small antenna system & radio propagation, and wireless communications & network performance evaluation.



**Alhareth Mohammed Zyoud** (M'18) received his bachelor degree in electrical engineering from Palestine Polytechnic University in 2006, his master and Ph.D. degrees in communication engineering from International Islamic University Malaysia in 2011 and 2017, respectively. He has authored or co-authored many research papers in international journals and conferences. His current research interests include RF modeling and simulation, 5G radio resource management, IOT, and rain attenuation analysis.